

Lecture 10: Weighted Interval Scheduling & Dynamic Programming

Date: August 31, 2026
Instructor: Aniket Basu Roy
Scribe: Shubham Bhilare & Sarang Shastry

Overview. In this lecture, we introduce the *Dynamic Programming* (DP) design paradigm through the WEIGHTED INTERVAL SCHEDULING PROBLEM. While unweighted interval scheduling can be solved optimally using a simple greedy strategy (picking earliest finish times), adding job weights causes standard greedy heuristics to fail. We systematically analyze why simple greedy attempts fail, formalize the compatibility predecessor function $p(j)$, derive the fundamental DP recurrence relation, prove its correctness by induction, present efficient algorithms for computing the optimal value and reconstructing the optimal set of jobs, and establish the standard 7-step paradigm for dynamic programming.

1 Problem Formalization

In the classical INTERVAL SCHEDULING PROBLEM, we are given a collection of jobs with fixed start and finish times, and our goal is to select the maximum number of mutually compatible jobs. However, in real-world resource allocation problems, different tasks carry non-uniform values, profits, or weights. This leads to the WEIGHTED INTERVAL SCHEDULING PROBLEM.

Definition 1 (Instance). An instance of the WEIGHTED INTERVAL SCHEDULING PROBLEM consists of a set $J = \{I_1, I_2, \dots, I_n\}$ of n jobs. Each job I_i is defined by a triple (s_i, f_i, w_i) where:

- $s_i \in \mathbb{R}$ is the start time,
- $f_i \in \mathbb{R}$ is the finish time with $s_i < f_i$, and
- $w_i > 0$ (or $w(I_i)$) is the positive weight/value of job I_i .

Definition 2 (Job Compatibility). Two distinct jobs I_i and I_j are *compatible* (or non-overlapping), denoted $I_i \cap I_j = \emptyset$, if their execution intervals do not overlap, i.e., $f_i \leq s_j$ or $f_j \leq s_i$. A subset of jobs $J' \subseteq J$ is *compatible* if all distinct pairs $I_i, I_j \in J'$ are mutually compatible.

Objective. Find a compatible subset $J' \subseteq J$ that maximizes the total weight:

$$\max_{J' \subseteq J \text{ compatible}} w(J') = \sum_{I_i \in J'} w_i.$$

2 Failure of Greedy Heuristics

Before deriving a dynamic programming solution, we examine why standard greedy heuristics fail for the weighted variant:

- **Attempt 1: Maximum Weight First.** Select the job with the highest weight w_i and discard conflicting jobs.

Counterexample: Consider job I_1 with weight $w_1 = 10$ spanning $[1, 10]$, which conflicts with two mutually compatible jobs I_2 over $[1, 5]$ ($w_2 = 6$) and I_3 over $[6, 10]$ ($w_3 = 6$). Selecting I_1 gives a total weight of 10, whereas selecting $\{I_2, I_3\}$ yields weight 12.

- **Attempt 2: Efficiency Ratio First.** Sort jobs by weight-to-duration ratio $\frac{w_i}{f_i - s_i}$ in descending order and greedily pick non-overlapping jobs.

Counterexample: Job I_1 has interval $[1, 2]$ with $w_1 = 2$ (ratio 2). Job I_2 has interval $[1, 10]$ with $w_2 = 19$ (ratio 1.9). Picking I_1 blocks I_2 , producing total weight 2 instead of 19.

- **Attempt 3: Exhaustive Subset Search (2^J).** Consider all 2^n possible subsets of J , eliminate incompatible subsets, and return the maximum-weight subset.

While correct, evaluating all subsets takes exponential time $O(n2^n)$, which is computationally intractable for large n .

This breakdown demonstrates that greedy choices cannot make optimal global decisions without systematically evaluating subproblem trade-offs.

3 The Dynamic Programming Recurrence

3.1 Sorting and the Compatibility Predecessor $p(j)$

Without loss of generality (WLOG), assume the jobs in J are sorted in non-decreasing order of finish times:

$$f_1 \leq f_2 \leq \dots \leq f_n.$$

To express overlapping constraints formally, we introduce a helper function $p(j)$.

Definition 3 (Predecessor Function $p(j)$). For each job $j \in \{1, \dots, n\}$, $p(j)$ is defined as the largest index $i < j$ such that job I_i is compatible with job I_j :

$$p(j) = \max(\{0\} \cup \{i \in \{1, \dots, j-1\} \mid f_i \leq s_j\}).$$

If no job I_i ($i < j$) is compatible with I_j , we set $p(j) = 0$.

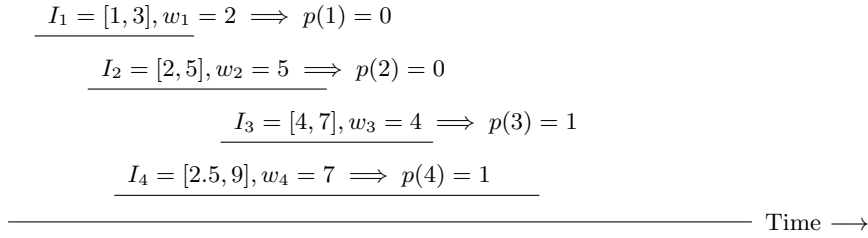


Figure 1: An example instance with $n = 4$ jobs sorted by finish times, illustrating interval spans, job weights w_i , and predecessor values $p(i)$.

3.2 Subproblem Definition and Case Analysis

For each $i \in \{0, 1, \dots, n\}$, let $J_i = \{I_1, I_2, \dots, I_i\}$ denote the subset consisting of the first i jobs in sorted order.

Definition 4 (Subproblem $OPT(i)$). Let $OPT(i)$ denote the maximum weight of any set of mutually compatible jobs chosen from J_i .

Consider an optimal solution $\mathcal{O}_i$ for the subproblem on J_i . We perform a case analysis on whether job I_i belongs to $\mathcal{O}_i$:

- **Case 1** ($I_i \notin \mathcal{O}_i$): If job I_i is not selected in $\mathcal{O}_i$, then $\mathcal{O}_i$ is an optimal solution for J_{i-1} . Therefore, the maximum weight obtainable is $OPT(i-1)$.
- **Case 2** ($I_i \in \mathcal{O}_i$): If job I_i is selected in $\mathcal{O}_i$, then no job in $\{p(i)+1, \dots, i-1\}$ can be in $\mathcal{O}_i$ because all such jobs overlap with I_i . Thus, the remaining jobs selected for $\mathcal{O}_i$ must be drawn from $J_{p(i)}$. To maximize total weight, the remaining jobs must form an optimal solution for $J_{p(i)}$, yielding weight $w_i + OPT(p(i))$.

Since these two cases are exhaustive, $OPT(i)$ is the maximum of both choices, yielding the recurrence relation:

$$OPT(i) = \begin{cases} 0 & \text{if } i = 0, \\ \max(OPT(i-1), w_i + OPT(p(i))) & \text{if } 1 \leq i \leq n. \end{cases} \quad (1)$$

4 Proof of Correctness

Theorem 1 (Correctness of Recurrence). *For every $i \in \{0, 1, \dots, n\}$, $OPT(i)$ defined by Recurrence (1) equals the maximum total weight of a compatible set of jobs chosen from J_i .*

Proof. We prove the theorem by strong mathematical induction on i .

Base Case ($i = 0$): $J_0 = \emptyset$. The only subset is empty with weight 0. Recurrence (1) gives $OPT(0) = 0$, which is correct.

Inductive Hypothesis: Assume Recurrence (1) correctly computes $OPT(k)$ for all $0 \leq k < i$.

Inductive Step: Consider an optimal solution $\mathcal{O}_i \subseteq J_i$ with total weight $w(\mathcal{O}_i)$.

If $I_i \notin \mathcal{O}_i$, then $\mathcal{O}_i \subseteq J_{i-1}$. By definition, $w(\mathcal{O}_i) \leq OPT(i-1)$. Conversely, any compatible solution for J_{i-1} is valid for J_i , so $OPT(i) \geq OPT(i-1)$. Hence, $w(\mathcal{O}_i) = OPT(i-1)$.

If $I_i \in \mathcal{O}_i$, then $\mathcal{O}_i \setminus \{I_i\}$ is a compatible set of jobs drawn from $J_{p(i)}$. By the inductive hypothesis, $w(\mathcal{O}_i \setminus \{I_i\}) \leq OPT(p(i))$, which gives $w(\mathcal{O}_i) \leq w_i + OPT(p(i))$. Conversely, adding I_i to an optimal set for $J_{p(i)}$ forms a valid compatible set for J_i , so $OPT(i) \geq w_i + OPT(p(i))$. Hence, $w(\mathcal{O}_i) = w_i + OPT(p(i))$.

Since $\mathcal{O}_i$ either excludes or includes I_i , $OPT(i) = \max(OPT(i-1), w_i + OPT(p(i)))$. By induction, $OPT(i)$ is correct for all $0 \leq i \leq n$. $\square$

5 Algorithms and Execution Trace

5.1 Iterative Dynamic Programming Algorithm

Recurrence (1) translates directly into an iterative bottom-up table-filling algorithm.

Algorithm 1: WEIGHTED-INTERVAL-SCHEDULING(s, f, w)

Input: Arrays $s[1 \dots n]$, $f[1 \dots n]$, $w[1 \dots n]$ of start times, finish times, and weights

Output: Maximum total weight $OPT(n)$ of a compatible set of jobs

- 1 Sort jobs by increasing finish times such that $f_1 \leq f_2 \leq \dots \leq f_n$
 - 2 Compute $p(i)$ for all $i \in \{1, \dots, n\}$ using binary search
 - 3 $OPT[0] \leftarrow 0$
 - 4 **for** $i \leftarrow 1$ **to** n **do**
 - 5 $OPT[i] \leftarrow \max(OPT[i-1], w_i + OPT[p(i)])$
 - 6 **return** $OPT[n]$
-

5.2 Concrete Execution Trace

We trace Algorithm 1 on the $n = 4$ job instance shown in Figure 1:

- Job I_1 : $w_1 = 2$, $p(1) = 0$
- Job I_2 : $w_2 = 5$, $p(2) = 0$
- Job I_3 : $w_3 = 4$, $p(3) = 1$
- Job I_4 : $w_4 = 7$, $p(4) = 1$

Evaluating values sequentially:

- $OPT[0] = 0$
- $i = 1$: $OPT[1] = \max(OPT[0], w_1 + OPT[p(1)]) = \max(0, 2 + 0) = 2$
- $i = 2$: $OPT[2] = \max(OPT[1], w_2 + OPT[p(2)]) = \max(2, 5 + 0) = 5$
- $i = 3$: $OPT[3] = \max(OPT[2], w_3 + OPT[p(3)]) = \max(5, 4 + OPT[1]) = \max(5, 4 + 2) = 6$
- $i = 4$: $OPT[4] = \max(OPT[3], w_4 + OPT[p(4)]) = \max(6, 7 + OPT[1]) = \max(6, 7 + 2) = 9$

The optimal value is $OPT[4] = 9$.

i	w_i	$p(i)$	$OPT[i-1]$	$w_i + OPT[p(i)]$	$OPT[i]$
0	—	—	—	—	0
1	2	0	0	$2 + 0 = 2$	2
2	5	0	2	$5 + 0 = 5$	5
3	4	1	5	$4 + 2 = 6$	6
4	7	1	6	$7 + 2 = 9$	9

Table 1: Dynamic programming table computation for the $n = 4$ job instance.

6 Reconstructing the Optimal Solution Set

Algorithm 1 computes the maximum weight $OPT(n)$, but does not explicitly output the set of selected jobs. Storing job subsets explicitly during execution would require $O(n^2)$ space. Instead, we reconstruct the optimal subset S by backtracking through the filled table OPT in $O(n)$ time (Algorithm 2).

Algorithm 2: FIND-SOLUTION(i, OPT, w, p)

Input: Index i , filled array OPT , weight array w , predecessor array p

Output: Subset $S \subseteq \{1, \dots, i\}$ of mutually compatible jobs yielding weight $OPT(i)$

```

1 if  $i = 0$  then
2   return  $\emptyset$ 
3 if  $w_i + OPT[p(i)] > OPT[i-1]$  then
4   return FIND-SOLUTION( $p(i), OPT, w, p$ )  $\cup \{I_i\}$ 
5 else
6   return FIND-SOLUTION( $i-1, OPT, w, p$ )

```

Backtracking Trace on Sample Instance:

1. Start at $i = 4$: $w_4 + OPT[p(4)] = 7 + OPT[1] = 9 > OPT[3] = 6 \implies$ Include I_4 , recurse on $p(4) = 1$.
2. At $i = 1$: $w_1 + OPT[p(1)] = 2 + OPT[0] = 2 > OPT[0] = 0 \implies$ Include I_1 , recurse on $p(1) = 0$.
3. At $i = 0$: Base case reached. Return $\emptyset$.

The optimal job subset is $J^* = \{I_1, I_4\}$ with total weight $w_1 + w_4 = 2 + 7 = 9$.

7 Complexity Analysis and the 7-Step DP Paradigm

7.1 Running Time and Space Complexity Analysis

We analyze the runtime across three distinct phases:

- Preprocessing:** Sorting n jobs by finish time takes $O(n \log n)$ time. Computing $p(i)$ for all $i \in \{1, \dots, n\}$ using binary search over finish times takes $O(\log n)$ per job, totaling $O(n \log n)$ time.
- Filling Table:** The memo table OPT contains $n + 1$ entries. Computing each entry takes $O(1)$ time. Thus, table evaluation takes $O(n)$ time.
- Postprocessing (Backtracking):** FIND-SOLUTION makes at most n recursive steps, spending $O(1)$ time per step, taking $O(n)$ time in total.

Claim 1 (Total Complexity). *The overall running time of the algorithm is $O(n \log n)$, and the auxiliary space complexity is $O(n)$ to store $OPT[0 \dots n]$ and $p[1 \dots n]$.*

7.2 The 7-Step Dynamic Programming Paradigm

Dynamic programming algorithms in this course follow a unified 7-step blueprint:

1. **Subproblem Definition:** Describe $OPT(i)$ in plain English and define all indices.
2. **Recurrence Relation:** Express $OPT(i)$ mathematically in terms of smaller subproblems.
3. **Proof of Correctness:** Prove the recurrence mathematically via induction and case analysis.

4. **Base Cases:** State initial seed conditions (e.g., $OPT(0) = 0$).
5. **Target Solution Mapping:** Specify which state answers the original query (e.g., $OPT(n)$).
6. **Algorithm & Reconstruction:** Write pseudo-code for iterative evaluation and backtracking.
7. **Complexity Analysis:** Bound preprocessing, memo-filling, and postprocessing runtimes and space.

Reference. Kleinberg, J., & Tardos, E. (2006). *Algorithm Design*, Chapter 6 (Dynamic Programming). Pearson/Addison-Wesley.